

TryHackMe - Intro To Malware Analysis

Malware

The word malware is derived from the term MALicious softWARE. Therefore, any software that has a malicious purpose can be considered malware. Malware is further classified into different categories based on its behavior.

Static Analysis

When malware is analyzed without being executed, it is called Static Analysis. In this case, the different properties of the PE file are analyzed without running it. Similarly, in the case of a malicious document, exploring the document's properties without analyzing it will be considered Static Analysis. Examples of static analysis include checking for strings in malware, checking the PE header for information related to different sections, or looking at the code using a disassemble. We will look at some of these techniques later in the room.

Malware often uses techniques to avoid static analysis. Some of these techniques use obfuscation, packing, or other means of hiding its properties. To circumvent these techniques, we often use dynamic analysis.

Dynamic Analysis

Malware faces a dilemma. It has to execute to fulfil its purpose, and no matter how much obfuscation is added to the code, it becomes an easy target for detection once it runs.

Static analysis might provide us with crucial information regarding malware, but sometimes that is not enough. We might need to run the malware in a controlled environment to observe what it does in these cases. Malware can often hide its properties to thwart Static Analysis. However, in most of those cases, Dynamic Analysis can prove fruitful. Dynamic analysis techniques include running the malware in a VM, either in a manual fashion with tools installed to monitor the malware's activity or in the form of sandboxes that perform this task automatically.

Malware, however, often uses techniques to prevent an analyst from performing dynamic analysis. Since most dynamic analysis is performed in a controlled environment, most methods to bypass dynamic analysis include detecting the environment in which it is being run.

Therefore, in these cases, the malware uses a different, benign code path if it identifies that it is being run in a controlled environment.

Advanced Malware Analysis

Advanced malware analysis techniques are used to analyze malware that evades basic static and dynamic analysis. For performing advanced malware analysis, disassemblers and debuggers are used. Disassemblers convert the malware's code from binary to assembly so that an analyst can look at the instructions of the malware statically. Debuggers attach to a program and allow the analyst to monitor the instructions in malware while it is running. A debugger allows the analyst to stop and run the malware at different points to identify interesting pieces of information while also providing an overview of the memory and CPU of the system

Basic Static Analysis

Examining the file type

Though often the file type of malware is visible in the file extension and is obvious, sometimes malware authors try to trick users by using misleading file extensions. In such scenarios, it is helpful to know how to find the actual file type of a file without depending on file extensions. In Linux, we can find the file type of a file using the `file` command.

Examining Strings

Another really important command that provides us with useful information about a file is the `strings` command. This command lists down the strings present in a file.

Calculating Hashes

File Hashing provides us with a fixed-size unique number that identifies a file. A File Hash can therefore be considered a unique identifier for a file, similar to Social Security Numbers or National Identification Numbers used for the citizens of a country. Hashing is an important concept in malware analysis. It can be used as an identifier for specific malware. As we will see later in this task, this identifier can then be shared with other analysts or searched online for information sharing purposes. Please note that a single bit of difference in two files will result in different hashes, so changing the hash of a file is as simple as changing one bit in it. Commonly, `md5sum`, `sha1sum` and `sha256sum` hashes are used for file hashing. We can calculate file hashes by using a simple command in Linux, as shown below for the `md5sum` hash:

```
md5sum <filename>
```

The PE File Header

PE Header

The PE File Header contains the metadata about a Portable Executable file. This data can help us find a lot of helpful information to help us in our analysis.

Imports/Exports

A PE file seldom contains all the code that it needs to run on a system on its own. Most of the time, it re-uses code provided by the Operating System. This is done to use less space and leverage the framework the Operating System has laid to perform tasks instead of re-inventing the wheel. Imports are such functions that the PE file imports from outside to perform different tasks.

For example, if a developer wants to Query a Windows Registry value, they will import the [RegQueryValue](#) function provided by Microsoft instead of writing the code themselves. It is understood that this function will be present on any Windows machine on which the developer's code is going to run, so it does not need to be included in the PE file itself. Similarly, any PE file export functions are exposed to other binaries that can use that function instead of implementing it themselves. Exports are generally associated with Dynamically-Linked libraries (DLL files), and it is not typical for a non-DLL PE file to have a lot of exports.

Since most PE files use the Windows API to perform the bulk of their jobs, a PE file's imports provide us with crucial information on what that PE file will do. It becomes evident that a PE file that is importing the InternetOpen function will communicate with the internet, a URLDownloadToFile function shows that a PE file will download something from the internet, and so on. Names of Windows APIs are generally intuitive and self-explanatory. However, we can always consult Microsoft Documentation to verify the purpose of a particular Windows function.

Sections:

Another useful piece of information available in the PE file header is the information about sections in the PE file. A PE file is divided into different sections which have different purposes. Although the sections in a PE file depend on the compiler or packer used to compile or pack the binary, the following are the most commonly seen sections in a PE file.

- **.text:** This Section generally contains the CPU instructions executed when the PE file is run. This section is marked as executable.
- **.data:** This Section contains the global variables and other global data used by the PE file.

- **.rsrc:** This Section contains resources that are used by the PE file, for example, images, icons, etc.

Basic Dynamic Analysis

Caution!

Dynamic analysis requires running live malware samples that can be destructive. It is highly recommended that you perform malware analysis in an isolated Virtual Machine. You can create a clean snapshot of your Virtual Machine before performing any malware analysis and revert it to start from a clean state again after every analysis. Don't perform malware analysis on a live machine not purpose-built for malware analysis.

Introduction to Sandboxes

Sandbox is a term borrowed from the military. A sandbox is a box of sand, as the name suggests, modeling the terrain where an operation has to take place, in which a military team dry runs their scenarios to identify possible outcomes. In malware analysis, a sandbox is an isolated environment mimicking the actual target environment of a malware, where an analyst runs a sample to learn more about it. Malware analysis sandboxes heavily rely on Virtual Machines, their ability to take snapshots and revert to a clean state when required.

Construction of a sandbox

For malware analysis using sandboxes, the following considerations make the malware analysis effective:

- Virtual Machine mimicking the actual target environment of the malware sample
- Ability to take snapshots and revert to clean state
- OS monitoring software, for example, Procmon, ProcExplorer or Regshot, etc.
- Network monitoring software, for example, Wireshark, tcpdump, etc.
- Control over the network through a dummy DNS server and webserver.
- A mechanism to move analysis logs and malware samples in and out of the Virtual Machine without compromising the host (Be careful with this one. If you have a shared directory with your malware analysis VM that remains accessible when running malware, you might risk malware affecting all files in your shared directory)

Open Source Sandboxes

Though it is good to understand what a good sandbox is made of, building a sandbox from scratch is not always necessary. One can always set up Open Source Sandboxes. These sandboxes provide the framework for performing basic dynamic analysis and are also customizable to a significant extent to help those with a more adventurous mindset.

Cuckoo's Sandbox

[Cuckoo's sandbox](#) is the most widely known sandbox in the malware analysis community. It was developed as part of a Google Summer of Code project in 2010. It is an open-source project that you will often see deployed in SOC environments and with enthusiasts' home labs. Advantages of Cuckoo's sandbox include huge community support, easy-to-understand documentation, and lots of customizations. You can deploy it on your network and let the community signatures guide you into identifying which files are malicious and which are benign because of the vast corpus of community signatures that come with it.

Cuckoo's sandbox has been archived, and an update is pending. It also doesn't support Python 3, making it obsolete right now. However, all is not lost because we have alternatives.

CAPE Sandbox

[CAPE Sandbox](#) is a little more advanced version of Cuckoo's sandbox. It supports debugging and memory dumping to support the unpacking of packed malware (We will learn more about packing and unpacking in the advanced malware analysis module). Though beginners can use this sandbox, advanced knowledge is required for making full use of it. A community version of this sandbox is available online, which can be used to test run it before installing. CAPE Sandbox is so far actively developed and supports Python 3.

Online Sandboxes:

Setting up and maintaining a sandbox can be a time-consuming task. Keeping that in view, online sandboxes can be of great help. Some of the most commonly used online sandboxes are as follows:

- [Online Cuckoo Sandbox](#)
- [Online CAPE Sandbox](#)
- [Any.run](#)
- [Intezer](#)
- [Hybrid Analysis](#)

Anti-analysis techniques

Packing and Obfuscation:

Malware authors often use packing and obfuscation to make an analyst's life difficult. A packer obfuscates, compresses, or encrypts the contents of malware. These techniques make it difficult to analyze malware statically. Specifically, a packed malware will not show important information when running a string search against it

Sandbox evasion:

As we have seen previously, we can always run a sample in a sandbox to analyze it. In many cases, that might help us analyze samples that evade our basic static analysis techniques. However, malware authors have some tricks up their sleeves that hamper that effort. Some of these techniques are as follows:

- **Long sleep calls:** Malware authors know that sandboxes run for a limited time. Therefore, they program the malware not to perform any activity for a long time after execution. This is often accomplished through long sleep calls. The purpose of this technique is to time out the sandbox.
- **User activity detection:** Some malware samples will wait for user activity before performing malicious activity. The premise of this technique is that there will be no user in a sandbox. Therefore there will be no mouse movement or typing on the keyboard. Advanced malware also detects patterns in mouse movements that are often used in automated sandboxes. This technique is designed to bypass automated sandbox detection.
- **Footprinting user activity:** Some malware checks for user files or activity, like if there are any files in the MS Office history or internet browsing history. If no or little activity is found, the malware will consider the machine as a sandbox and quit.
- **Detecting VMs:** Sandboxes run on virtual machines. Virtual machines leave artifacts that can be identified by malware. For example, some drivers installed in VMs being run on VMWare or Virtualbox give away the fact that the machine is a VM. Malware authors often associate VMs with sandboxes and would terminate the malware if a VM is detected.